

Staying in the Driver's Seat

Maintaining code ownership in the age of AI coding assistants

! Required Reading

This guide is required reading for all Rashid Lab members using AI coding tools. Read it before starting your first Claude Code session, and revisit it periodically.

[Download the PDF](#)

Introduction: The Tool Dependency Trap

AI coding assistants like Claude Code, GitHub Copilot, and Cursor have fundamentally changed how we write software. They make us faster, help us explore unfamiliar languages, and reduce the friction of boilerplate. But there's a quiet cost that many developers are only now recognizing: the gradual erosion of deep understanding.

If you've ever found yourself unable to make progress during an outage, unsure how a module you "wrote" actually works, or nervous about explaining your own code in a review—you're not alone. This guide offers concrete strategies for using AI tools effectively while maintaining genuine ownership of your work.

⚠ The Core Tension

AI tools are most dangerous not when they give wrong answers, but when they give right answers you don't understand. The goal is not to use AI less—it's to use it in ways that keep you learning.

Part 1: Foundational Principles

Before diving into specific habits, internalize these principles. They apply whether you're a seasoned researcher or a student writing your first project.

1. **Think First, Then Generate.** Before prompting an AI tool, spend a few minutes thinking about the approach yourself. Write pseudocode, sketch a diagram, or jot notes on which files are involved. Use the AI to execute your plan—not to make the plan for you.

i A statistical illustration: Correct code, wrong question

The dangerous output in statistics isn't a missing package — it's a model that does what AI suggested but answers a different question.

AI can produce	Human must decide	Failure if outsourced
Regression code	What estimand the question requires	Correct model, wrong question
Covariate-adjusted regression	Which covariates are pre- vs. post-treatment	Adjusted away the treatment effect by conditioning on a mediator
Imputation code	Which missingness mechanism is plausible (MCAR / MAR / MNAR)	Imputed under MCAR when MAR/MNAR is more credible; inference understates uncertainty
Simulation	What data-generating process matters	Simulation validates the wrong world

The middle column is where the chain of judgment lives. It's the column AI doesn't help with — not because it can't, but because the choices in that column are what the work is *for*.

2. **Read Everything That Ships.** Never commit code you can't explain. Treat AI output like a pull request from a junior developer: review every line, question the design choices, and understand the “why” behind each decision.
3. **Own the Architecture.** It's fine to delegate syntax and boilerplate. It's not fine to delegate system design. You should always be able to draw your system on a whiteboard and explain how the pieces connect.
4. **Maintain Your Mental Model.** A mental model is your internal map of how your codebase works—what depends on what, where state lives, how data flows. If you can't trace a request through your system from input to output, your mental model has gaps.
5. **Invest in Fundamentals Continuously.** AI tools change rapidly; fundamentals don't. Data structures, algorithms, system design, and debugging skills are the bedrock that lets you evaluate AI output critically.

Part 2: Daily Habits and Workflows

These are the concrete, repeatable practices that keep your understanding sharp. Pick the ones that resonate and build them into your routine.

Habit 1: The Pre-Prompt Pause

Before typing a prompt, spend 2–5 minutes writing down your intent in a scratch file or notebook. Answer three questions:

- What am I trying to accomplish? (The goal, not the implementation.)
- What’s my best guess for how to do it? (Even if rough or wrong.)
- What are the tricky parts? (Edge cases, performance concerns, unknowns.)

Then prompt the AI with your plan as context. This ensures you’re steering the tool rather than being steered by it. Compare the AI’s approach to yours—the differences are where learning happens.

Habit 2: The Code Review Mindset

When AI generates code, switch into code reviewer mode. Don’t just check that it works—ask yourself:

- Could I have written this myself? If not, what concept am I missing?
- Is this the approach I would have chosen? Why or why not?
- What assumptions is this code making about inputs, environment, or state?
- Are there edge cases it’s not handling?
- Is this over-engineered or under-engineered for the actual need?

If you find code you don’t understand, stop and learn the underlying concept before accepting it. **This is the single most important habit in the guide.**

Habit 3: The Decision Log

Maintain a lightweight markdown file (`DECISIONS.md`) in every project. Each entry captures:

- The date and what decision was made.
- Why that approach was chosen over alternatives.
- What trade-offs were accepted.
- Which components are affected.

This log serves three purposes: it forces you to articulate your understanding, it's invaluable during outages when you can't ask the AI, and it helps teammates (or future-you) understand the reasoning behind the code.

💡 Template: DECISIONS.md

```
## 2026-03-11: Switched from REST to GraphQL for the dashboard API
```

```
Reason: The dashboard requires fetching nested data across 4 entities. REST  
required 6 round trips; GraphQL collapses this to 1.
```

```
Trade-off: Added complexity in the resolver layer. Team needs to learn GraphQL  
schema design.
```

```
Affected: /api/graphql, /components/Dashboard, /lib/resolvers
```

Habit 4: Unplugged Sessions

Dedicate 1–2 hours per week to coding without AI assistance. Use this time to:

- Debug something manually using print statements, a debugger, or log files.
- Read through a module or file you haven't touched in a while.
- Refactor a small piece of code by hand.
- Write a new utility function from scratch.

This is deliberate practice. Like a musician practicing scales or an athlete doing drills, it keeps your core skills from atrophying. It also builds confidence that you can function when tools are unavailable.

Habit 5: Narrate as You Go

When working with AI tools, keep a running narration (mental or written) of what's happening at the system level. Instead of thinking "I asked Claude to fix the bug and it worked," think "The bug was a race condition in the cache invalidation. Claude suggested using a mutex, which works because the shared state is only accessed from two goroutines."

The narration forces you to process the AI's work through your own understanding. If you can't narrate what happened, that's a signal to slow down and learn.

Habit 6: Red-team Your Output

Before accepting an AI-assisted result, argue against it. Four questions in particular — the questions a hostile reviewer asks first:

- **What estimand did this actually answer?** AI gives you what you literally asked for. The estimand question makes sure that matches what your protocol or your science actually needs.
- **What diagnostic would make me distrust this?** A concrete falsification criterion. If you can't name one, you don't have a model — you have a fitting procedure.
- **What did the AI not suggest?** The AI is generative within its training distribution; alternatives outside that distribution are the human's responsibility.
- **What would I do if the result were reversed?** A pre-mortem on the conclusion. If you'd report the work the same way regardless of which direction the finding went, you don't have a directional finding — you have a fitting procedure dressed as a finding.

For regulated work, these become a sponsor-reviewable checklist. Many SAPs already include something like this; making it AI-aware just means asking the questions in light of how AI helped produce the analysis.

Modes of AI Use

Before moving from daily habits to project-level practices, a vocabulary worth pausing on. How you prompt AI determines what kind of work it does for you. Three modes are worth distinguishing:

Executor mode — “*Write a function that...*” — AI produces the work. Code, drafts, format conversions. This is the default; the output goes into your deliverable.

Tutor mode — “*Quiz me on this method before showing code*” — AI teaches you something rather than doing it for you. The output is understanding; the work product still has to be yours.

Critic mode — “*Argue against this model choice; list two alternatives I should consider*” — AI challenges your work. The output is calibration; you're generating disagreement to stress-test your thinking.

Default AI is executor. The other two modes are *designed* — they don't happen unless you prompt them.

💡 Worked example: three modes, one statistical situation

Same question: *choosing between OLS, GLS, and robust regression for data with some heteroskedasticity.*

Mode	Prompt	What you get
Executor	<i>“Use GLS with HC3 standard errors, write the code.”</i>	Code that runs. You never asked whether GLS was the right call.
Tutor	<i>“Walk me through the assumptions of each — when does each model fail? Quiz me before showing code.”</i>	A primer + retrieval practice. Assumptions encoded, not just applied.
Critic	<i>“Argue for OLS despite the heteroskedasticity. What residual pattern would I need to see?”</i>	Three scenarios where OLS would still be defensible. Stress-test of your default.

Same AI, same data, three different conversations. The mode is a lever you control.

One important caveat: **critic-mode output is itself delegated judgment** — it needs the same review as code. If the AI argues against your choice unconvincingly, that’s not evidence your model is right; it’s evidence the AI didn’t argue well.

Part 3: Project-Level Practices

Architecture Documentation

Maintain a living document (ARCHITECTURE.md) that describes your system at a high level. Include:

- A diagram or description of major components and how they interact.
- The data model and how state flows through the system.
- Key design decisions and their rationale (links to your decision log).
- External dependencies and why each one was chosen.

Update this document whenever the system changes significantly. Writing it yourself—not asking AI to generate it—is the point. The act of writing is the act of understanding.

The Explain-It-Back Test

After AI helps you build a feature, close the AI tool and try to explain the implementation to a rubber duck, a colleague, or a voice memo on your phone. If you get stuck, you’ve

identified a gap in your understanding. Go back and fill it—read the code, read documentation, experiment.

For students, this can be formalized: after completing an AI-assisted assignment, write a short paragraph explaining the approach and why it works. The paragraph itself isn't the point—the thinking required to write it is.

Periodic Code Walkthroughs

Schedule regular walkthroughs of your codebase, either alone or with collaborators. Pick a module and trace its behavior end to end. Ask questions like:

- What happens when this function receives unexpected input?
- Where does this data come from, and where does it end up?
- If this component fails, what's the blast radius?

These walkthroughs are especially important for code that was heavily AI-assisted, since it's the code most likely to contain assumptions you haven't internalized.

Part 4: For Educators and Team Leads

If you're teaching students or leading a team, these strategies help create a culture of understanding alongside AI adoption.

Classroom Strategies

The Two-Pass Assignment. Have students complete assignments in two passes. In the first pass, they work without AI assistance and submit their attempt, even if incomplete. In the second pass, they use AI tools to improve, complete, or refactor their work. They submit both versions along with a reflection document explaining what the AI contributed and what they learned from the differences.

Explain-Your-Code Assessments. Supplement code submissions with oral or written explanations. Ask students to walk through their code, explain their choices, and answer “what if” questions. This is not about catching cheating—it's about reinforcing the understanding that should accompany working code.

Debugging Exercises. Give students intentionally broken code and ask them to find and fix the bugs without AI tools. Debugging is the skill most at risk of atrophy and also one of the best ways to build deep understanding of how code actually executes.

Team Practices

Code Review Culture. Establish that AI-generated code receives the same (or more) scrutiny in code review as human-written code. Reviewers should ask: “Can you walk me through this?” If the author can’t, the PR isn’t ready.

Shared Decision Logs. Make the decision log a team practice, not just a personal one. When decisions are visible and discussed, they become shared understanding rather than knowledge trapped in one person’s head (or one AI conversation’s history).

Rotation and Pairing. Rotate who works on different parts of the codebase. Pair programming—especially pairing someone who used AI heavily on a feature with someone reviewing it fresh—is an excellent way to distribute understanding.

Part 5: Red Flags and Self-Assessment

Periodically check yourself against these warning signs. If several apply, it’s time to adjust your workflow.

- ☐ You can’t make meaningful progress on your project when AI tools are unavailable.
- ☐ You accept AI-generated code without reading it carefully because “it works.”
- ☐ You can’t explain how a recent feature you built actually works under the hood.
- ☐ You’ve stopped learning new concepts because AI can handle things you don’t understand.
- ☐ Your debugging strategy is “paste the error into the AI” with no hypothesis of your own.
- ☐ You couldn’t draw your system architecture on a whiteboard from memory.
- ☐ You feel anxious rather than mildly inconvenienced when AI tools go down.
- ☐ You’ve started skipping documentation because “I can just ask the AI later.”

Scoring: 0–1 checked: You’re in great shape. 2–4: Some habits need adjustment. 5+: Time for a deliberate reset—start with the unplugged sessions and decision log.

Part 6: The Right Mental Model for AI Tools

The GPS Analogy

Think of AI coding assistants like GPS navigation. GPS didn’t make people worse drivers, but it did make many people worse at knowing where they are. The people who still have a sense of direction are the ones who glance at the map occasionally, notice landmarks, and pay attention to the route—even while GPS is guiding them.

Your goal is to be the developer who uses AI to go faster on a path you understand, not to navigate a path you couldn’t walk alone.

The Spectrum of Delegation

Not all tasks require the same level of understanding. Here's a helpful way to think about what's safe to delegate heavily versus what requires your full attention:

Safe to Delegate	Delegate with Review	Stay Hands-On
Boilerplate and scaffolding	Business logic implementation	Architecture and system design
Syntax and language idioms	Test writing and edge cases	Security-critical code
Format conversions	Refactoring existing code	Performance-critical paths
Documentation drafts	API integration	Core algorithms you must understand
Regex and string patterns	Database queries	Debugging complex issues

Conclusion

The developers and researchers who will thrive in the AI era aren't the ones who reject these tools, nor the ones who surrender entirely to them. They're the ones who maintain a deliberate practice of understanding—who treat AI as a powerful collaborator but never stop being the person who knows *why* the code works, not just *that* it works.

The habits in this guide are not about slowing down. They're about building the kind of understanding that makes you faster in the long run: faster at debugging when things break, faster at adapting when requirements change, and faster at evaluating whether the AI's suggestion is the right one.

Start small. Pick one or two habits from this guide and practice them consistently. The goal is not perfection—it's awareness. As long as you're paying attention to your own understanding, you're on the right track.

Quick-Start Checklist

- ☐ Start a `DECISIONS.md` file in your current project today.
- ☐ Before your next AI prompt, write down your approach first.
- ☐ Schedule one 90-minute unplugged coding session this week.
- ☐ Next time AI generates code, do a line-by-line review before committing.
- ☐ Try the explain-it-back test on the last feature you built with AI help.